06 22313 Computational Tools for Modelling and Analysis

Fitting Models to Data

Dr Iain Styles (I.B.Styles@bham.ac.uk)

School of Computer Science, University of Birmingham

October 19 2015

Introduction

Least Squares Fitting

Linear Models

Finding the Best Fit

Weighted Linear Least Squares

Fitting Nonlinear Models

Outline

Introduction

Least Squares Fitting

Linear Models

Finding the Best Fit

Weighted Linear Least Squares

Fitting Nonlinear Models

Fitting Models to Data

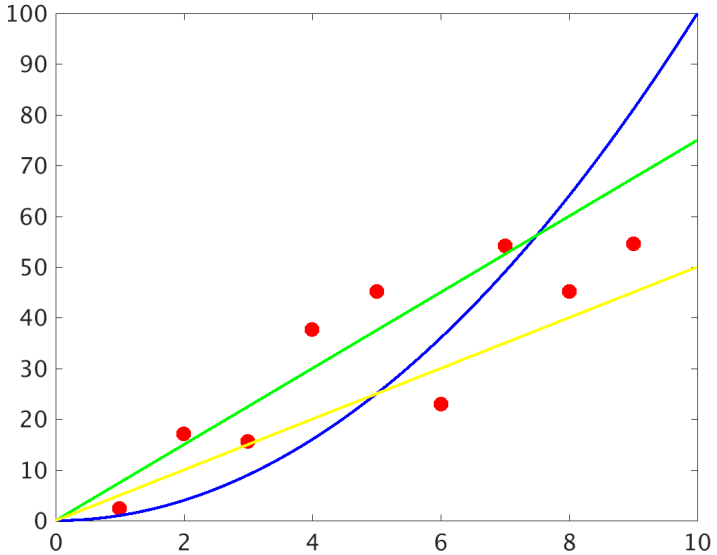
- A problem of great interest to cardiovascular scientists is to understand the dynamics of blood flow during the cardiac cycle
  - Where are the cells, how fast are they moving, are they accelerating?
- In a tracking experiment, you may be able to measure the positions of particles at set time points
- How do you calculate the velocity, acceleration, etc from the positions?
- You fit a **model** of the motion to the points, for example:

$$s = ut + \frac{1}{2}at^2,$$

where the unknowns are u, the initial velocity and a, the acceleration. These are **free parameters** that can be chosen so that the model describes the data well.

Types of Model

- How we find the optimal values of the unknowns depends on the type of model
- We will consider models of three different classes
  - Linear models:
    $y = mx + c$,
    $s = ut + \frac{1}{2}at^2$
  - Linear models with measurement error.
  - Nonlinear models:
    $y = a + b\sin(cx)$,
    $y = axe^{-a^2x}$



Least Squares Fitting

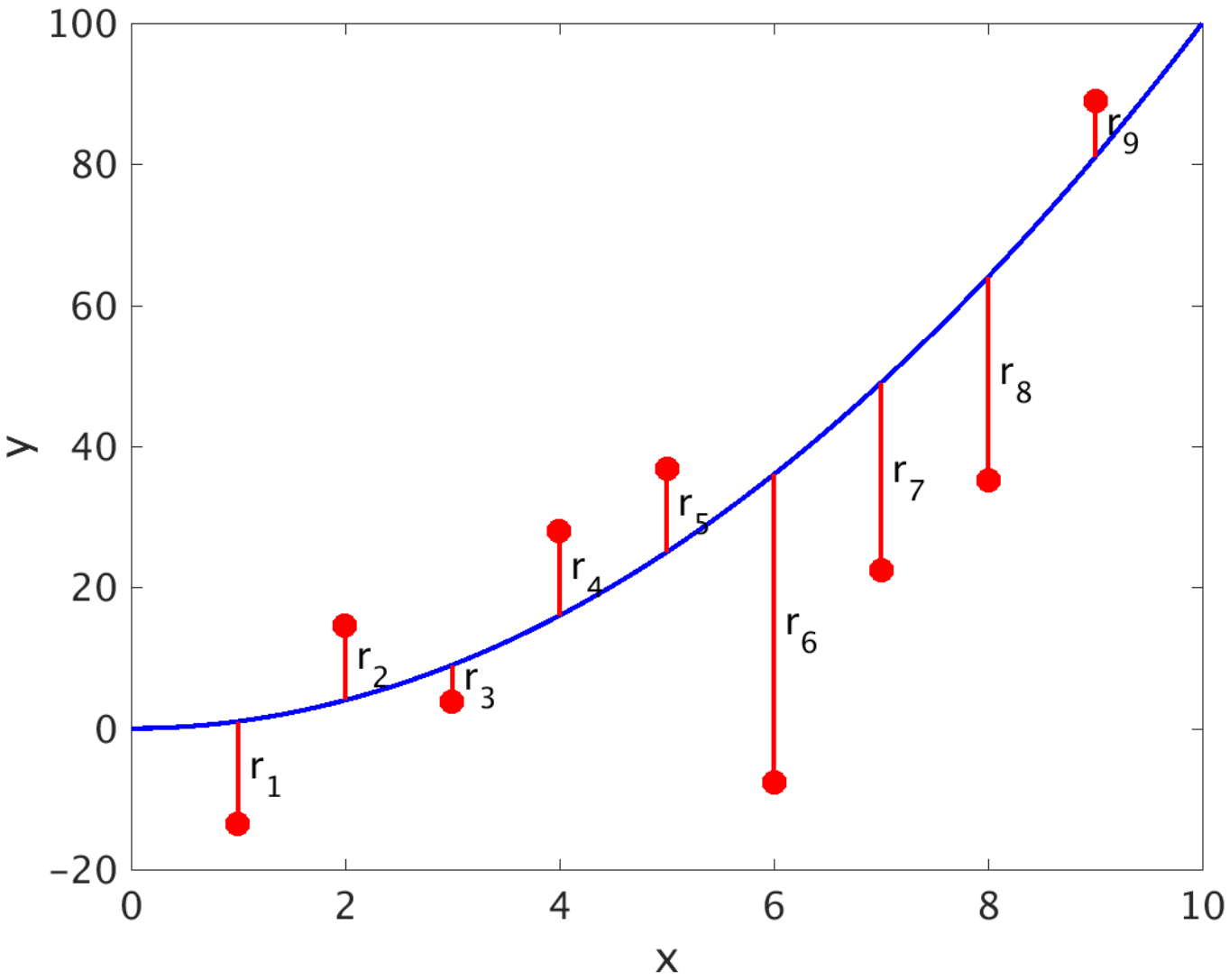
- Minimise "difference" between model and data
- Data: $\mathcal{D} = \{(x_1, y_1), \ldots, (x_n, y_n)\}$
- Model: $y = f(\mathbf{p}, x)$
- Difference (**residual**): $r_i(\mathbf{p}) = y_i - f(\mathbf{p}, x_i)$
- In terms of which the **least squares error** is

$$e(\mathbf{p}) = \sum_{i=1}^{n} r_i^2 = \mathbf{r}^{\mathrm{T}}\mathbf{r} \tag{1}$$

- Goal: find the value of $\mathbf{p} = \mathbf{p}^*$ that minimises $e(\mathbf{p})$:

$$\mathbf{p}^* = \arg\min_{\mathbf{p}} e(\mathbf{p}) \tag{2}$$

Least Square Fitting



Linear Models

- Not just straight lines!
- Any function linear in the unknown parameters.

$$f(\mathbf{p}, x) = p_1 f_1(x) + \cdots + p_m f_m(x) = \sum_{j=1}^{m} p_j f_j(x) \quad (3)$$

- The residuals are therefore

$$r(\mathbf{p}, x_i) = y_i - \sum_{j=1}^{m} p_j f_j(x_i) \quad (4)$$

- Treating $\mathbf{p}$ as a column vector and defining matrix f $\mathbf{F}$ as $F_{ij} = f_j(x_i)$ we have

$$\mathbf{r} = \mathbf{y} - \mathbf{F}\mathbf{p} \quad (5)$$

where $\mathbf{y} = (y_1; \ldots; y_n)$.

Example: Quadratic Model

- Quadratic model $f(\mathbf{p}, x) = p_1 x^2 + p_2 x + p3$.
- Each row of $\mathbf{F}$ corresponds to a data point
- Each column corresponds to a term in the model:

$$\begin{aligned} \mathbf{r} &= \mathbf{y} - \mathbf{F}\mathbf{p} \\ \begin{pmatrix} r_1 \\ r_2 \\ \vdots \\ r_n \end{pmatrix} &= \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} - \begin{pmatrix} x_1^2 & x_1 & 1 \\ x_2^2 & x_2 & 1 \\ & \vdots & \\ x_n^2 & x_n & 1 \end{pmatrix} \begin{pmatrix} p_1 \\ p_2 \\ p_3 \end{pmatrix}. \end{aligned}$$

- Our least-squares error can therefore be written

$$e(\mathbf{p}) = \sum_i r_i^2 = \mathbf{r}^{\mathrm{T}}\mathbf{r} = (\mathbf{y} - \mathbf{F}\mathbf{p})^{\mathrm{T}} (\mathbf{y} - \mathbf{F}\mathbf{p}) \qquad (6)$$

Solving for the Best Fit

- "Best fit" $\rightarrow$ find $\mathbf{p} = \mathbf{p}^*$ that minimises $e(\mathbf{p})$.
- Differentiate $e(\mathbf{p})$ with respect to $\mathbf{p}$ and set to zero:

$$\mathbf{F}^{\mathrm{T}}\mathbf{y} - \mathbf{F}^{\mathrm{T}}\mathbf{F}\mathbf{p}^* = 0$$

- Simultaneous equations: solve by direct inversion:

$$\mathbf{p}^* = \left(\mathbf{F}^{\mathrm{T}}\mathbf{F}\right)^{-1}\mathbf{F}^{\mathrm{T}}\mathbf{y}$$

- or in Matlab using Gaussian elimination through the \ operator:
- Why not solve $\mathbf{y} = \mathbf{F}\mathbf{p}$ for $\mathbf{p}$ directly?
  - Guarantees on specific solution (least squares)
  - Incorporate extra information into $e(\mathbf{p})$

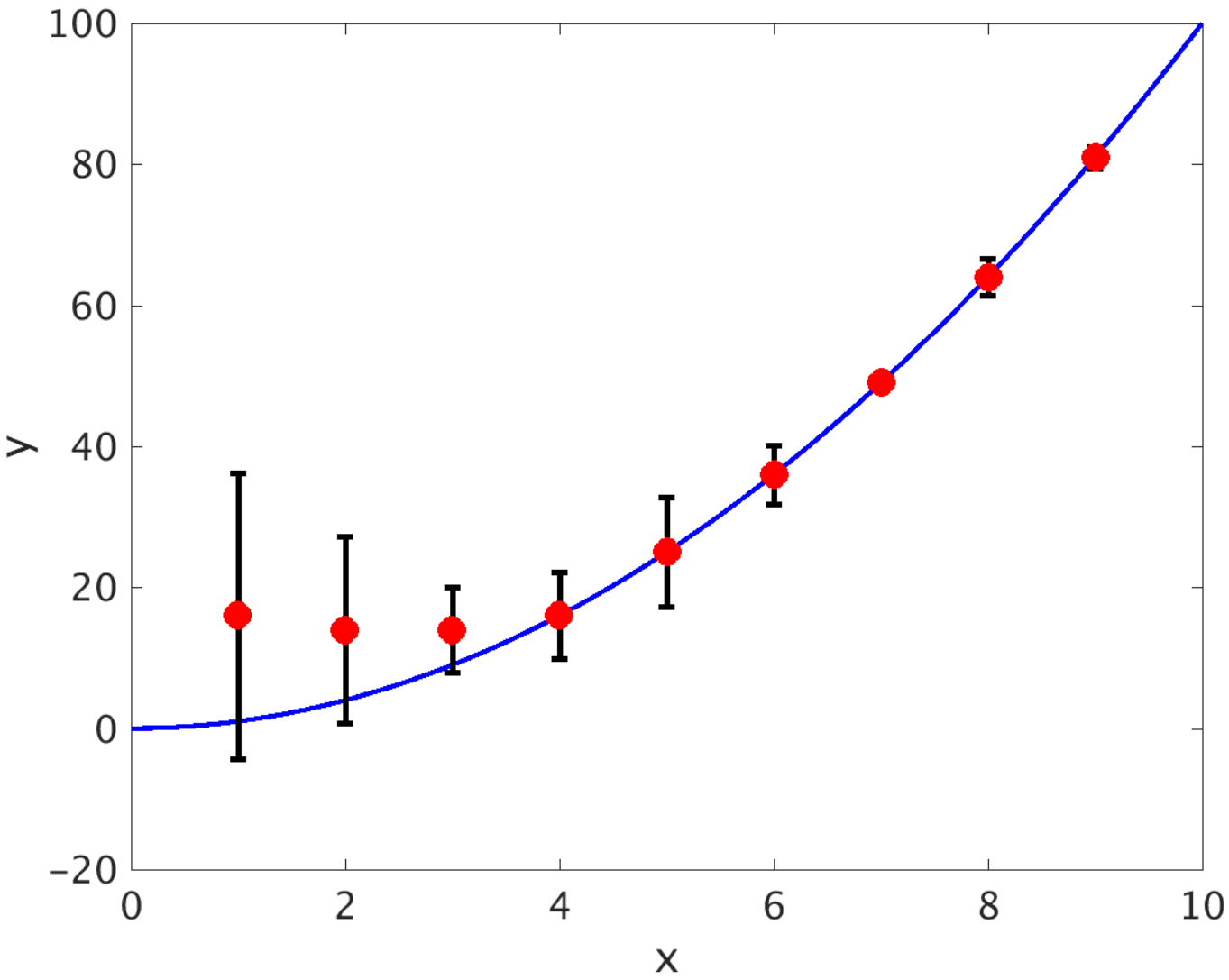
Weighted Least Squares

- Experimental data $\rightarrow$ multiple repeats. How do we deal with this?
- Fit models to the "average" but does the spread (variance) matter?
- Given less influence to "noisy/uncertain" data by modifying $e(\mathbf{p})$

$$e(\mathbf{p}) = \sum_{i=1}^{n} w_i r_i^2$$

- Set $w_i = \frac{1}{\mathrm{var}(y_i)}$
- "Certain" points influence the fit more strongly than "uncertain" ones.

Weighted Least Squares



Weighted Least Squares

- Covenient to define matrix

$$\mathbf{\Sigma} = \mathrm{diag}\left(\mathrm{var}(y_1), \mathrm{var}(y_1), \ldots, \mathrm{var}(y_n)\right)$$

- Special case of the **covariance matrix**
- The error becomes

$$e(\mathbf{p}) = \mathbf{r}^{\mathrm{T}}\mathbf{\Sigma}^{-1} = (\mathbf{y} - \mathbf{F}\mathbf{p})^{\mathrm{T}}\,\mathbf{\Sigma}^{-1}\,(\mathbf{y} - \mathbf{F}\mathbf{p})$$

- Note that $\mathbf{\Sigma}^{-1}$ can be done by inverting the diagonal elements
- Differentiate and set to zero

$$\mathbf{F}^{\mathrm{T}}\mathbf{\Sigma}^{-1}\mathbf{y} - \mathbf{F}^{\mathrm{T}}\mathbf{\Sigma}^{-1}\mathbf{F}\mathbf{p}^{*} = 0, \tag{7}$$

- Solve as before

$$\mathbf{p}^{*} = (\mathbf{F}^{\mathrm{T}}\mathbf{\Sigma}^{-1}\mathbf{F})^{-1}\mathbf{F}^{\mathrm{T}}\mathbf{\Sigma}^{-1}\mathbf{y} \tag{8}$$

Fitting Nonlinear Models

- What about non-linear models like $y(x) = a \sin bx$?
- No matrix representation (non-linear!)
- Cannot minimise least square error directly.
- Instead, focus on **reducing** the error iteratively
- We will turn non-linear problems into locally linear ones.

Linearising Non-linear Problems

Add figure in here that shows how to Taylor-expand a 1d function

- Taylor Expansion

$$y(x + \Delta x) = y(x) + \frac{\mathrm{d}y}{\mathrm{d}x}\Delta x$$

Linearising Non-linear Problems

- Taylor expansion of our fitting function

$$\begin{aligned} f(x_i, \mathbf{p} + \mathbf{\Delta p}) &\approx f(x_i, \mathbf{p}) + \sum_{j=1}^{m} \frac{\partial f(x_i)}{\partial p_j} \Delta p_j \\ &= f(x_i, \mathbf{p}) + \sum_{j=1}^{m} J_{ij} \delta p_j \end{aligned}$$

- Writing $f_i(\mathbf{p}) = f(x_i, \mathbf{p})$ give $\mathbf{f}(\mathbf{p} + \mathbf{\Delta p}) = \mathbf{f}(\mathbf{p}) + \mathbf{J}\mathbf{\Delta p}$
- The residual is $\mathbf{r}(\mathbf{p} + \mathbf{\Delta p}) = \mathbf{y} - \mathbf{f}(\mathbf{p}) - \mathbf{J}\mathbf{\Delta p}$
- And the error becomes

$$e(\mathbf{p} + \mathbf{\Delta p}) = (\mathbf{y} - \mathbf{f}(\mathbf{p}) - \mathbf{J}\mathbf{p})^{\mathrm{T}} (\mathbf{y} - \mathbf{f}(\mathbf{p}) - \mathbf{J}\mathbf{p}) .$$

- Minimise this $\rightarrow$ change in $\mathbf{p}$ that maximally reduces the error
- Differentiate and set to zero:

$$\left(\mathbf{J}^{\mathrm{T}}\mathbf{J}\right) \mathbf{\Delta p}^{*} = \mathbf{J}^{\mathrm{T}} \left(\mathbf{y} - \mathbf{f}(\mathbf{p})\right)$$

- Solve, repeat until finished!

Stabilising the Optimisation

- Common modification:

$$\left(\mathbf{J}^{\mathrm{T}}\mathbf{J} + \lambda\,\mathrm{diag}(\mathbf{J}^{\mathrm{T}}\mathbf{J})\right)\boldsymbol{\Delta}\mathbf{p}^{*} = \mathbf{J}^{\mathrm{T}}\left(\mathbf{y} - \mathbf{f}(\mathbf{p})\right)$$

- Without this modification, the parameter vector will change quickly along directions with steep gradients
- But slowly along shallow gradients.
- Modification increases movement along shallow gradients and thus faster convergence.
- "Damping factor" λ controls the extent of this.
- Modified iteratively to give the best results.
- The full algorithm is named after its inventors: **Levenberg-Marquadt**

The Levenberg-Marquadt Algorithm

```
Data: Data x, y; Initial parameter guess p_0; Function
      f(x, p); Jacobian Jac(x, p); initial values of λ, ν.
Result: p, e(p), parameters that minimmise the least
        squares error, the error.
% Evaluate the error at the initial parameter
  value
p ← p_0;
e ← (y − f(x, p)^T (y − f(x, p);
% Iterate until convergence
Converged ← False;
while Converged == False do
    % Calculate the Jacobian
    J ← Jac(x, p);
    % Calculate update for different values of λ
      until we improve the error
    LambdaSet ← false;
    while LambdaSet==false do
        % Calculate Δp for λ = λ_0 and λ = λ_0/ν
        Δ_1 = (J^T J + λ diag(J^T J)) \ J^T (y − f(p))%;
        Δ_2 = (J^T J + (λ/ν) diag(J^T J)) \ J^T (y − f(p));
        % Calculate the error at the new
          parameter values
        e_1 ← (y − f(x, p + Δ_1)^T (y − f(x, p);
        e_2 ← (y − f(x, p + Δ_2)^T (y − f(x, p);
        if e_1 < e then
            % Set new values of p and e, storing
              old value as e'
            p ← p + Δ_1;
            e' ← e;
            e ← e_1;
            % Current λ is OK
            LambdaSet ← true;
        else if e_2 < e then
            % Set new values of p and e, storing
              old value as e'
            p ← p + Δ_2;
            e' ← e;
            e ← e_2;
            λ ← λ/ν;
            % Current λ is OK
            LambdaSet ← true;
        else
            λ ← λ/ × nu;
    end
    if e'/e < TerminationCriterion then
        Converged ← true
    end
end
```